

HACKATHON

Plataforma Digital de Atendimento SUS

Relatório técnico do projeto

Arquitetura • Segurança • Dados • Mensageria • Infraestrutura • Testes

Equipe: Luis • Thiago • Juliana • Gilmar • Danilo

1. Visão geral do projeto

A plataforma propõe centralizar etapas do atendimento ao paciente do SUS em um único ambiente digital, conectando pacientes, médicos, atendentes e administradores com regras de acesso bem definidas.

O sistema começa pelo cadastro dos usuários e permite que pacientes acompanhem seu atendimento, realizem agendamentos de consultas e exames, consultem resultados, pareceres, receitas e documentos. Médicos poderão analisar resultados remotamente, emitir pareceres, solicitar novos exames, emitir ou renovar receitas e gerar documentos relacionados ao atendimento.

O objetivo é reduzir etapas manuais e deslocamentos desnecessários, melhorar a rastreabilidade do atendimento e concentrar informações clínicas e administrativas em uma estrutura única e organizada.

Princípio central da solução

Separar o dado técnico produzido pelo exame da interpretação clínica do médico. O resultado do exame é persistido independentemente do parecer médico, permitindo histórico, autoria, rastreabilidade e evolução futura do sistema.

1.1 Objetivos

- Centralizar os dados e o histórico de atendimento do paciente.
- Facilitar o agendamento, remarcação e cancelamento de consultas e exames.
- Disponibilizar resultados de exames de imagem e laboratoriais.
- Permitir análise remota e emissão de parecer por médicos autorizados.
- Permitir emissão e renovação de receitas e geração de documentos médicos.
- Notificar usuários sobre eventos importantes do atendimento.
- Aplicar controle de acesso por perfil utilizando Spring Security.

2. Funcionalidades do MVP

Módulo	Escopo
Usuários e segurança	Cadastro de usuários; autenticação; associação de roles; controle de acesso por perfil.
Consultas	Agendamento, remarcação, cancelamento e acompanhamento.
Exames	Solicitação médica, agendamento, registro da realização e acompanhamento.
Resultados	Resultados de imagem ou laboratoriais, com parâmetros, unidades e faixas de referência.
Parecer médico	Registro da interpretação médica vinculada ao resultado do exame.

Módulo	Escopo
Receitas	Emissão, renovação e consulta de receitas e seus itens.
Documentos	Atestados, laudos, relatórios, encaminhamentos e declarações.
Notificações	Avisos sobre consultas, exames, resultados, pareceres, receitas e documentos.

2.1 Perfis e níveis de acesso

Role	Permissões principais
ADMINISTRADOR	Acesso global; cadastro e gerenciamento de usuários, perfis, cadastros auxiliares, consultas e exames.
ATENDENTE	Atendimento administrativo; acesso a dados necessários do paciente e gestão de agendamentos. Não emite parecer, receita ou solicitação médica.
MEDICO	Acesso clínico; consulta histórico e resultados, solicita exames, emite pareceres, receitas e documentos.
PACIENTE	Acesso às próprias informações, consultas, exames, resultados, pareceres, receitas, documentos e notificações.

3. Segurança e controle de acesso

A autenticação e autorização serão implementadas com Spring Security. O usuário recebe uma ou mais roles, e as permissões são aplicadas nos endpoints e casos de uso de acordo com o perfil autenticado.

Decisão de modelagem

Administrador e Atendente não precisam de entidades próprias. Eles são representados por USUARIO associado às roles ADMINISTRADOR ou ATENDENTE. MEDICO e PACIENTE possuem entidades próprias porque armazenam dados específicos do domínio.

Entidade	Responsabilidade	Relacionamento
USUARIO	Identidade, credenciais e dados comuns do acesso.	N:N com ROLE via USUARIO_ROLE.
ROLE	Perfis de autorização.	ADMINISTRADOR, ATENDENTE, MEDICO e PACIENTE.
USUARIO_ROLE	Tabela associativa entre usuários e roles.	FK usuario_id e role_id.

Entidade	Responsabilidade	Relacionamento
PACIENTE	Dados específicos do paciente.	1:1 com USUARIO.
MEDICO	Dados específicos do médico, CRM e especialidade.	1:1 com USUARIO.

Exemplo de proteção de operação clínica:

```
@PreAuthorize("hasRole('MEDICO')")
```

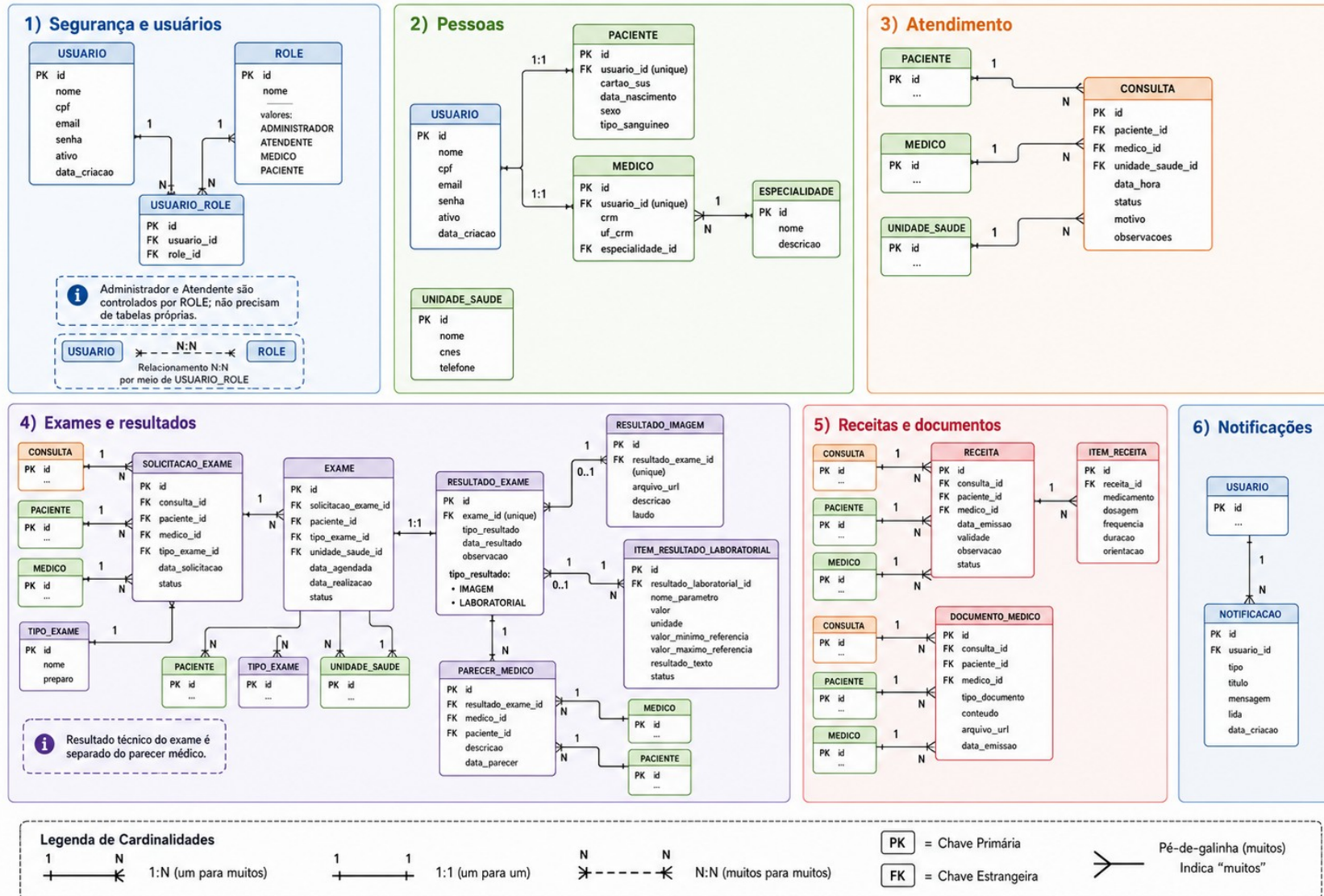
4. Modelo de dados

A modelagem foi organizada por responsabilidade. Consultas, solicitações, exames, resultados, pareceres, receitas, documentos e notificações são entidades independentes para reduzir acoplamento e facilitar a evolução do domínio.

Domínio	Entidades principais
Segurança	USUARIO, ROLE, USUARIO_ROLE
Pessoas	PACIENTE, MEDICO, ESPECIALIDADE, UNIDADE_SAUDE
Atendimento	CONSULTA
Exames	TIPO_EXAME, SOLICITACAO_EXAME, EXAME
Resultados	RESULTADO_EXAME, RESULTADO_IMAGEM, RESULTADO_LABORATORIAL, ITEM_RESULTADO_LABORATORIAL
Análise clínica	PARECER_MEDICO
Receitas	RECEITA, ITEM_RECEITA
Documentos	DOCUMENTO_MEDICO
Notificações	NOTIFICACAO

5. Diagrama Entidade-Relacionamento

Diagrama Entidade-Relacionamento – Sistema SUS + Spring Security



6. Fluxo de exames e resultados

O módulo de exames separa claramente a solicitação médica, a realização do exame, o resultado técnico e o parecer clínico. Essa separação evita que informações de naturezas diferentes sejam misturadas e permite auditoria e histórico mais confiáveis.

1. O médico cria uma SOLICITACAO_EXAME vinculada ao paciente, à consulta e ao TIPO_EXAME.
2. O EXAME representa a execução/agendamento daquela solicitação em uma UNIDADE_SAÚDE.
3. Após a realização, é criado um RESULTADO_EXAME.
4. O resultado técnico pode ser do tipo IMAGEM ou LABORATORIAL.
5. O médico analisa o resultado e registra um PARECER_MEDICO independente.
6. A partir do atendimento, o médico pode emitir RECEITA e DOCUMENTO_MEDICO.

6.1 Resultado de imagem

RESULTADO_IMAGEM é utilizado para raio-X, tomografia, ressonância, ultrassonografia e outros exames que gerem arquivos ou imagens. O registro pode armazenar uma URL/referência de arquivo, descrição e laudo técnico.

6.2 Resultado laboratorial

RESULTADO_LABORATORIAL representa o conjunto do exame. Cada parâmetro do resultado é armazenado em ITEM_RESULTADO_LABORATORIAL, permitindo registrar valor, unidade, faixa de referência e status.

Parâmetro	Resultado	Referência	Status
Hemoglobina	14,2 g/dL	13,0 a 17,0	NORMAL
Glicemia	135 mg/dL	70 a 99	ACIMA_REFERENCIA
HIV	Não reagente	Não reagente	NORMAL

O campo resultadoTexto permite representar respostas não numéricas, como POSITIVO, NEGATIVO, NÃO REAGENTE ou tipos sanguíneos. Valores mínimo e máximo são aplicados quando a referência é quantitativa.

7. Pareceres, receitas e documentos médicos

7.1 Parecer médico

PARECER_MEDICO registra a interpretação clínica do resultado, identificando o médico responsável, o paciente, o resultado analisado, a descrição e a data do parecer. O resultado técnico não é alterado pelo parecer.

7.2 Receita e itens

RECEITA representa o documento emitido pelo médico e pode conter vários ITEM_RECEITA. Cada item registra medicamento, dosagem, frequência, duração e orientação.

7.3 Documento médico

DOCUMENTO_MEDICO representa documentos gerados durante o atendimento, como atestados, laudos, relatórios, encaminhamentos e declarações. O documento fica associado ao médico, paciente e consulta.

Histórico do paciente

No MVP, não é necessário criar uma tabela HISTORICO. O histórico pode ser construído cronologicamente a partir de consultas, solicitações, exames, resultados, pareceres, receitas e documentos. Isso evita duplicação de dados.

8. Arquitetura e tecnologias

A solução utilizará Clean Architecture para manter regras de negócio desacopladas de frameworks e detalhes de infraestrutura. A organização pode ser feita em camadas de domínio, aplicação, infraestrutura e apresentação.

Camada	Responsabilidade
Domain	Entidades, enums, regras de negócio e contratos de repositório.
Application	Casos de uso, DTOs, mapeadores e orquestração.
Infrastructure	JPA/Hibernate, PostgreSQL, Spring Security, Kafka, configurações e integrações.
Presentation	Controllers REST, entrada HTTP e exposição da API.

Tecnologia	Uso
Java	Linguagem principal
Spring Boot / Spring Framework	API e infraestrutura da aplicação
Spring Security	Autenticação e autorização
Jakarta Bean Validation	Validação de entrada e regras declarativas
Spring Data JPA + Hibernate	Persistência e ORM
PostgreSQL	Banco de dados relacional
Apache Kafka	Mensageria e eventos
Docker / Docker Compose	Padronização do ambiente
JUnit + Mockito	Testes unitários e mocks
Swagger / OpenAPI	Documentação da API

Tecnologia	Uso
Flyway	Versionamento das migrations
Observabilidade (opcional)	Logs, métricas e monitoramento

9. Mensageria e notificações

Kafka será utilizado para desacoplar eventos de negócio da geração de notificações. A ação principal publica um evento e um consumidor responsável por notificações registra a mensagem para o usuário.

Evento sugerido	Disparo
ConsultaAgendadaEvent	Nova consulta criada.
ConsultaRemarcadaEvent	Alteração de data/hora.
ConsultaCanceladaEvent	Cancelamento da consulta.
ExameAgendadoEvent	Agendamento de exame.
ResultadoExameDisponivelEvent	Resultado técnico disponibilizado.
ParecerMedicoCriadoEvent	Médico conclui análise.
ReceitaEmitidaEvent	Nova receita disponibilizada.
DocumentoMedicoEmitidoEvent	Novo documento médico disponível.

10. Divisão do projeto

A divisão abaixo foi ajustada para acompanhar os módulos do modelo entidade-relacionamento e reduzir sobreposição de responsabilidades durante o desenvolvimento.

Responsável	Frente	Entregas principais
Luis	Arquitetura e segurança	Modelo ER; Clean Architecture; USUARIO, ROLE, USUARIO_ROLE; PACIENTE; MEDICO; ESPECIALIDADE; TIPO_EXAME; configuração base do Spring Security.
Thiago	Consultas, exames e receitas	CONSULTA; SOLICITACAO_EXAME; EXAME; resultados de imagem e laboratório; RECEITA; ITEM_RECEITA; visualização do histórico.

Responsável	Frente	Entregas principais
Juliana	Clínico e qualidade	PARECER_MEDICO; DOCUMENTO_MEDICO; testes automatizados/unitários; apoio em Swagger e observabilidade.
Gilmar	Infraestrutura	Docker; Docker Compose; PostgreSQL; Flyway; apoio a armazenamento/geração de arquivos.
Danilo	Mensageria e entrega	Kafka; NOTIFICACAO; consumidores/produtores de eventos; relatório final; vídeo e apoio à apresentação.

11. Escopo prioritário e evoluções

11.1 Prioridades do MVP

- Cadastro, autenticação e controle por roles.
- Cadastro de pacientes, médicos, especialidades e tipos de exame.
- Agendamento e gestão de consultas.
- Solicitação e agendamento de exames.
- Registro de resultado de imagem e laboratorial.
- Parecer médico sobre o resultado.
- Emissão de receita e documentos médicos.
- Notificações orientadas a eventos.
- Histórico básico do paciente derivado dos registros existentes.

11.2 Evoluções futuras

- Telemedicina e integração com sistemas oficiais de saúde.
- Assinatura digital de receitas e documentos.
- Suporte a DICOM para exames de imagem.
- Integração com laboratórios e serviços externos.
- Notificações push, e-mail e canais externos.
- Auditoria detalhada de alterações e trilha de acesso.
- Dashboard administrativo e aplicativo mobile.

12. Conclusão

A proposta cria uma base consistente para digitalizar partes importantes do atendimento, mantendo separação clara entre identidade, atendimento, resultado técnico, análise médica e documentos clínicos. O modelo entidade-relacionamento sustenta o fluxo do MVP e mantém espaço para evoluções futuras sem exigir reestruturação completa do domínio.

A combinação de Spring Security, PostgreSQL, JPA/Hibernate, Kafka, Docker, Flyway e testes automatizados fornece uma base técnica adequada para o Hackathon, enquanto a Clean

Architecture ajuda a organizar responsabilidades e reduzir o acoplamento entre regras de negócio e infraestrutura.

12.1 Critérios de sucesso do MVP

- Autenticação funcionando com autorização por roles.
- Fluxo demonstrável de consulta, solicitação de exame e resultado.
- Suporte a resultado de imagem e resultado laboratorial estruturado.
- Parecer médico, receita e documento vinculados corretamente ao paciente.
- Ambiente reproduzível com Docker e banco versionado por Flyway.
- Notificações desacopladas por eventos quando a mensageria estiver habilitada.

Resultado esperado do Hackathon

Entregar uma API funcional e demonstrável em que usuários autenticados atuem conforme suas roles, pacientes possam acompanhar seus atendimentos e exames, e médicos possam analisar resultados, registrar pareceres e emitir receitas e documentos.

13. Links Úteis

Repositório de código e gravações do hackathon.

Repositório GitHub

<https://github.com/lh-borges/Fiap-Hackton-SistemSaude-Grupo-32>

Vídeo · Pitch de Apresentação

<https://www.youtube.com/watch?v=dUrBcF15nv8>

Vídeo · Demonstração do MVP

<https://youtu.be/WIOXiWBTkg0>

14. Aprendizados e Próximos Passos

14.1 Principais aprendizados da equipe

Spring Modulith impõe fronteiras de módulo em tempo de build, tornando a arquitetura modular verificável e não apenas documentada.

A separação entre entidade de domínio e entidade JPA exige disciplina, mas simplifica a evolução de cada camada de forma independente.

Eventos de domínio internos (Spring ApplicationEvent) e externos (Kafka) podem coexistir com um flag de feature, facilitando testes locais sem broker.

ArchUnit automatiza a verificação da regra de dependência, eliminando revisões manuais de arquitetura no code review.

O gate de cobertura de 80% no perfil quality força a cobertura a crescer junto com o código, e não apenas no final do projeto.

14.2 Próximos passos

- Telemedicina e integração com sistemas oficiais de saúde.
- Assinatura digital de receitas e documentos.
- Suporte a DICOM para exames de imagem.
- Integração com laboratórios e serviços externos.
- Notificações push, e-mail e canais externos.
- Dashboard administrativo e aplicativo mobile.